HealthGuard AI

Complete Senior Software Engineer / GenAI Developer Interview Preparation Guide

Project Type	Healthcare GenAI / RAG / Voice Assistant / Patient Safety Chatbot
Goal	Safe, short, user-friendly guidance without diagnosis, prescriptions, or
Capabilities	Assessment, chatbot, voice input, document upload, RAG, reports, doctor
Interview Focus	Architecture, safety, RAG, memory isolation, NLP, voice, testing, deploy

This PDF is written for practical interview preparation. It explains the implemented project, design decisions, backend and frontend flows, GenAI architecture, production readiness, and cross-question answers.

1. Project Overview

HealthGuard AI is a healthcare assistant workspace for patients, doctors, admins, and compliance users. It allows patients to enter health details, ask health questions, use voice input, upload medical documents, generate educational reports, and prepare for doctor review.

The business problem is fragmented patient information and unclear symptom communication. The application organizes inputs and provides safe, short guidance while avoiding diagnosis, prescription, and dosage advice.

- Patient: assessment, chatbot, voice, upload, reports.
- Doctor/Dietician: review reports and add notes.
- Admin: manage trusted knowledge and citations.
- Compliance: audit and privacy controls.

2. Complete Tech Stack

```
Frontend       | HTML/CSS/JavaScript, React- | Responsive dashboard, chatbot, voice UI, mobil
Backend        | Python FastAPI              | Typed APIs, service layer, async-ready
LLM            | Gemini/OpenAI-ready abstrac | Provider swap, JSON contract, fail-closed
RAG            | Extraction, chunks, embeddi | Grounded answers from documents
Database       | SQLite fallback / PostgreSQ | Demo plus production persistence
Vector DB      | Local vectors / pgvector /  | Semantic search with user metadata
Auth           | Email OTP, sessions, SSO pa | Verified users and RBAC
Email          | SMTP no-reply               | Verification and password reset
PDF            | Built-in generator / Report | Reports and interview docs
Voice          | Web Speech API / ASR fallba | Natural voice questions
Monitoring     | Health endpoints, audit log | Operational readiness
CI/CD          | GitHub Actions/Jenkins-read | Tests, build, deploy, rollback
```

3. Complete Project Structure

```
healthguard-ai/
  backend/app/api/           FastAPI routes
  backend/app/schemas/       Pydantic contracts
  backend/app/services/      Auth, chat, NLP, RAG, LLM, report, PDF, speech, storage
  backend/app/db/            SQLite/PostgreSQL schema and connections
  backend/tests/             Safety, auth, RAG, memory, upload, report tests
  frontend/index.html        Dashboard, chatbot, voice, forms, reports
  frontend/app.js            API calls, memory keys, voice capture, UI behavior
  frontend/styles.css        Responsive healthcare UI and chatbot drawer
  docs/                      Documentation
  scripts/                   Utility and PDF generation scripts
  output/pdf/                Generated artifacts
  infra/                     Docker/deployment-ready assets
```

The architecture is layered so routes stay thin, schemas validate data, services own business logic, and storage/RAG/LLM integrations are isolated and testable.

4. System Architecture

The browser talks to FastAPI. FastAPI enforces auth/RBAC, routes work to services, stores data in the database, retrieves RAG context, calls the LLM abstraction when needed, and generates PDFs or audit logs.

```
Browser UI -> FastAPI Routes -> Auth/RBAC -> Services
  -> Chat + NLP + Safety -> RAG Retrieval -> LLM/Rules -> Chat Memory
  -> Assessment -> Report Engine -> PDF
  -> Upload -> Extract -> Chunk -> Embed -> Vector Store
  -> SMTP -> Audit Logs -> Health Checks
```

5. Backend Architecture

Routes handle HTTP, consent, auth, and orchestration. Services implement domain logic. Storage owns database queries. RAG and LLM layers isolate external systems. This separation improves testing and production maintainability.

- API routes
- Pydantic schemas
- Services layer
- Database/repository layer
- RAG layer
- LLM provider layer
- Security and monitoring layers

6. Frontend Architecture

The frontend includes login/register, email verification, dashboard, chatbot, voice assistant, assessment form, document upload, reports, doctor review, admin knowledge upload, and mobile responsive navigation.

- Textbox clears after Send
- Loading states
- Mobile hamburger menu
- Floating chatbot
- Voice states: Listening, Processing, Responding

7. Authentication and Authorization Flow

Users register, verify email OTP, log in, and access role-specific dashboards. Backend route protection is mandatory because frontend hiding is only UX.

- Patient, doctor, admin, compliance roles
- Session/JWT-ready handling
- Google SSO path
- User data isolation

8. Chatbot Functionality

The chatbot gives short 1-3 line answers, avoids repeated disclaimers, shows urgent safety only for red flags, stores user-specific memory, and scopes history by user_id and conversation_id.

- Stores user message immediately
- Fetches latest 5-10 messages
- Uses active conversation only
- Does not mix users or old conversations
- Current message is always included

```
Memory flow:
1. Save current user message
2. Fetch latest messages for user_id + conversation_id ORDER BY id DESC LIMIT 10
3. Reverse rows into chronological order
4. Build prompt with recent memory + current question + RAG context
5. Save assistant response
```

9. Voice Assistant Functionality

Voice input is lightweight chatbot mode. It uses continuous listening, interim transcripts, a manual Stop button, and 5-second silence detection. It sends only the final transcript to the chat API.

- Does not generate full reports
- Asks 3-4 human-style follow-ups
- Summarizes responses
- Gives precautions, diet, hydration, rest, and doctor-visit preparation
- Escalates red flags calmly

10. NLP Layer / Intent Understanding

The NLP layer runs before final response generation. It detects intent, symptoms, duration, severity, age, location, medicines, allergies, conditions, red flags, missing fields, and next action.

- Intent examples: symptom_guidance, medicine_question, lab_report_question, diet_question, emergency_symptom, general_health_question, better_suggestion_info_request
- Example answer: I need symptom, duration, severity, age, existing conditions, current medicines, allergies, recent reports, and red flags.

11. RAG Pipeline

File upload goes through validation, extraction, cleaning, chunking, embedding, vector storage, metadata storage, retrieval, reranking, prompt building, LLM/rules answer generation, citations, and user-specific isolation.

- Chunk size controls context granularity
- Overlap preserves boundary context
- top_k controls retrieval cost
- Similarity threshold reduces irrelevant context

12. RAG Data Sources

Uploadable sources include lab reports, discharge summaries, prescriptions, health reports, allergy records, diagnosis history, diet plans, BP/sugar logs, synthetic patient files, and legal public demo datasets such as Synthea/MIMIC demo.

- Markdown is good for clean text hierarchy
- JSON is good for structured lab/vitals data

13. Report Generation Flow

User completes assessment, mandatory fields and consent are validated, triage/risk scoring runs, RAG context is retrieved, a base safety report is built, Gemini may enhance it, and the result is saved and downloadable as PDF.

- Fail-closed if LLM enhancement fails
- Audit log records generation
- Doctor-ready questions and reminders included

14. Safety and Guardrails

HealthGuard does not diagnose, prescribe, or recommend dosage. Safety rules refuse medication changes, escalate red flags, use calm wording, and avoid scary language.

- Prompt injection prevention
- PHI/privacy protection
- Static disclaimer strategy
- Doctor consultation guidance

15. User Memory and Chat History Isolation

Memory is needed for follow-ups but must be scoped. The app uses user_id and conversation_id, never mixes users, and does not mix old conversations unless opened.

- Latest memory bug fix: save current message first
- Query latest rows DESC
- Reverse before LLM prompt
- Frontend active conversation stored per user

```
SELECT role, content, created_at
FROM chat_messages
WHERE user_id = :user_id AND conversation_id = :conversation_id
ORDER BY id DESC
LIMIT 10;
messages = list(reversed(messages))
```

16. Database Design

```
users                   | identity, email, role, verification
sessions                | active login records
email_verifications     | OTP and reset tokens
patient_profiles        | demographics and lifestyle
assessments             | structured health inputs
reports                 | generated report JSON and status
chat_conversations      | conversation owner and metadata
chat_messages           | role, content, sources, flags
rag_documents           | uploaded/admin documents
rag_chunks              | chunk text and embeddings
knowledge_documents     | admin trusted knowledge
audit_logs              | compliance events
doctor_reviews          | review notes, status, signature
```

17. API Design

```
POST /api/auth/register                 | Create user
POST /api/auth/login                    | Login
POST /api/auth/verify-email             | Verify OTP
GET /api/auth/sso/{provider}/start      | Start SSO
GET /api/auth/sso/{provider}/callback   | SSO callback
POST /api/reports/generate              | Generate report
GET /api/reports                        | List reports
GET /api/reports/{id}                   | Read report
GET /api/reports/{id}/download          | Download PDF
POST /api/chat/health-question          | Chatbot
POST /api/documents/upload              | Upload/index document
GET /api/health/rag                     | RAG health
POST /api/admin/knowledge               | Add knowledge
GET /api/admin/knowledge                | List knowledge
GET /api/doctor/reports/pending         | Doctor queue
POST /api/doctor/reports/{id}/review    | Doctor review
GET /api/health                         | App health
GET /api/health/db                      | DB health
GET /api/health/llm                     | LLM health
```

18. LLM Integration

LLM provider abstraction supports Gemini and OpenAI-ready providers. The LLM enhances structured safe output; it is not the primary safety engine. JSON response contracts, retries, token limits, and fail-closed behavior reduce risk.

19. Prompt Engineering

Prompts define system behavior, RAG context usage, concise chatbot answers, voice follow-ups, safety refusal, and JSON output. Prompt engineering prevents hallucination by constraining context and style.

20. Fine-Tuning vs Prompt Engineering

Prompt engineering changes instructions. Fine-tuning trains behavior. RAG supplies dynamic knowledge. Fine-tune repeated classifiers such as intent/safety/lab extraction, but do not fine-tune private patient records.

21. Handling Large File Uploads

For 5000 Excel files, save the file, create a job, enqueue with Redis/Celery, parse in workers using openpyxl read_only mode, batch insert rows, create RAG chunks, track progress, retry failures, and isolate users.

22. Async, Multithreading, and Multiprocessing

Async uses an event loop for I/O waits. Threads help blocking I/O. Multiprocessing handles CPU-heavy OCR/PDF parsing. HealthGuard can use async for LLM/DB calls, background threads for SMTP, and workers for OCR/Excel parsing.

23. Performance Optimization

Use async FastAPI, background workers, Redis caching, batch embeddings, metadata filters, top_k tuning, reranking, chunk optimization, smaller models for classification, larger models for final answers, token limits, streaming, and rate limiting.

24. Monitoring and Observability

Track API latency, P95 latency, error rate, token usage, cost/request, LLM latency, vector DB latency, RAG hit rate, hallucination rate, citation coverage, user feedback, queue length, and failed jobs.

- OpenTelemetry
- Prometheus
- Grafana
- CloudWatch/Azure Monitor
- ELK/Datadog

25. CI/CD Pipeline

Pipeline: linting, unit tests, integration tests, RAG evaluation, prompt regression tests, security scan, Docker build, deploy to dev, deploy to staging, manual approval, production deployment, smoke tests, rollback.

26. Testing Strategy

Tests cover auth, role guards, patient isolation, chat memory, latest question memory, RAG retrieval, prompt output, voice assistant, upload, report generation, LLM fail-closed, API health, and privacy flows.

27. Security and Privacy

Use auth, RBAC, user-specific records, patient-specific RAG chunks, no cross-user chat history, no personal Gmail in production, no-reply email with SPF/DKIM/DMARC, .env security, secret management, audit logs, PHI masking, and HTTPS.

28. Deployment

Local runs with Uvicorn. Production should use Docker, PostgreSQL, Redis, object storage, reverse proxy/HTTPS, ngrok for demo only, environment variables, health endpoints, backups, monitoring, and hardened auth/email setup.

29. Scenarios Implemented

Registration/email verification, login/dashboard, patient assessment, voice questions, chatbot short answer and follow-up mode, fever/rash, medication refusal, document upload/RAG indexing, report PDF, doctor review, admin knowledge, mobile menu, chat memory bug fix, and 5000 Excel upload design.

30. Interview Explanation

30-second explanation

HealthGuard AI is a healthcare GenAI workspace that lets patients ask safe health questions, use voice, upload reports, complete assessments, and generate doctor-ready educational reports. It uses FastAPI, RAG, NLP intent understanding, user-specific memory, RBAC, and safety guardrails.

2-minute explanation

The system includes a responsive dashboard, chatbot, voice assistant, document upload, RAG indexing, and report generation. The backend validates auth and consent, stores chat memory by user and conversation, runs NLP and safety checks, retrieves RAG context, and uses an LLM abstraction only when useful.

5-minute architecture explanation

Start with browser flows: auth, dashboard, assessment, chat, voice, upload, and reports. FastAPI routes enforce user and role checks. Services handle chat/NLP/safety/RAG/LLM/report/PDF/storage. Uploaded documents become user-scoped chunks. Reports use triage, risk scoring, RAG context, optional Gemini enhancement, fail-closed behavior, audit logs, and PDF generation.

Senior Engineer / GenAI role

As a Senior Engineer, emphasize modular architecture, testing, memory isolation, RBAC, performance, and production hardening. As a GenAI Developer, emphasize RAG, prompts, NLP extraction, safety guardrails, model abstraction, and evaluation.

31. Cross Questions and Answers

1. Why RAG?
RAG grounds answers in uploaded documents and approved knowledge without training private patient data into the model.

2. Why not only LLM?
LLMs can hallucinate. HealthGuard uses deterministic safety rules, NLP intent extraction, RAG context, and tests.

3. How do you avoid hallucination?
Use scoped retrieval, citations, safe base answers, refusal rules, and prompt regression tests.

4. How do embeddings work?
Chunks become numeric vectors. Similar vectors represent semantically related content.

5. What vector DB did you use?
The demo supports local embedding JSON and is ready for pgvector or Qdrant.

6. How does chunking work?
Documents are split into overlapping chunks with metadata for precise retrieval.

7. How do you handle latest records?
Fetch latest chat rows DESC, limit them, then reverse to chronological order.

8. How do you isolate user data?
Every chat, report, and RAG query filters by user_id and conversation_id or ownership.

9. How do you handle voice input?
Continuous speech recognition captures interim text and sends the final transcript to lightweight chat mode.

10. Why async?
Async improves throughput for I/O waits such as LLM, DB, SMTP, and vector search.

11. Async vs threading vs multiprocessing?
Async handles waiting I/O, threads handle blocking I/O, and multiprocessing handles CPU-heavy parsing/OCR.

12. How do decorators work?
Decorators wrap functions to add behavior such as auth checks, logging, caching, or rate limits.

13. How do you handle 5000 Excel uploads?
Save files, create jobs, process with Redis/Celery workers, use read_only mode, batch inserts, and progress tracking.

14. What metrics do you monitor?
Latency, P95, errors, token usage, cost, LLM latency, vector latency, RAG hit rate, queue length, and feedback.

15. How do you test RAG?
Use known documents, expected chunks, citation checks, isolation tests, and prompt regressions.

16. Prompt engineering vs fine-tuning?
Prompting changes instructions; fine-tuning trains behavior. RAG supplies dynamic knowledge.

17. How CI/CD works for GenAI?
Run unit tests, integration tests, RAG evals, prompt regressions, security scans, Docker build, deploy, and rollback.

18. How do you secure PHI?
Use auth, RBAC, owner filters, masked logs, secret management, HTTPS, and audit logs.

19. How do you handle LLM failure?
Fail closed for reports and use rules-based fallback for safe chat when possible.

20. What is fail-closed behavior?
Stop a risky operation instead of saving or returning incomplete unsafe output.

21. How do you reduce cost?
Short prompts, smaller classifiers, top_k tuning, caching, token limits, and avoiding unnecessary LLM calls.

22. How do you improve performance?
Batch embeddings, cache metadata, tune chunks, use workers, stream responses, and reduce tokens.

23. Why FastAPI?
It provides typed APIs, validation, OpenAPI docs, async readiness, and clean Python service structure.

24. Why Pydantic?
It validates data contracts and prevents malformed health data from entering service logic.

25. Why layered architecture?
It separates routes, schemas, services, storage, RAG, LLM, and security for testability.

26. What is the NLP layer?
It extracts intent, symptoms, duration, severity, age, medicines, allergies, conditions, and missing fields.

27. What intents are detected?
Symptom guidance, medicine question, lab report question, diet question, emergency symptom, and info request.

28. How are red flags detected?
Rules detect chest pain, breathing trouble, bleeding, confusion, fainting, and severe weakness.

29. Why no repeated disclaimer?
Repeated disclaimers hurt UX; safety appears when risk requires it and static guidance covers general limits.

30. How do you refuse medication?
The bot says it cannot recommend medication/dosage and asks for context or clinician review.

31. How do you handle dosage questions?
Dosage is refused because it needs clinician judgment and patient-specific context.

32. What is user-specific memory?
Recent messages from the current user_id and conversation_id only.

33. How was the memory bug fixed?
Current user message is saved first, latest rows are fetched DESC, and rows are reversed before prompt use.

34. How do you prevent old chats mixing?
Backend filters by conversation_id and frontend stores active conversation keys per user.

35. Can doctors see patient chats?
Not by default. Doctor workflow focuses on reviewable reports, not private chat memory.

36. What documents can users upload?
Lab reports, prescriptions, discharge summaries, scan notes, doctor notes, vitals, and allergy records.

37. What documents should not be uploaded?
IDs, bank files, resumes, unrelated photos, or non-health files.

38. How do you validate uploads?
Size/type checks, malware pattern scan, extraction checks, and audit logging.

39. How are documents indexed?
Extract text, clean, chunk, embed, store metadata, and link chunks to user_id.

40. What is metadata filtering?
Filtering retrieval by user_id, source type, document ID, category, and ownership.

41. Why citations?
They make answers auditable and reduce unsupported claims.

42. How choose chunk size?
Balance semantic completeness, retrieval precision, and token cost.

43. What is overlap?
Shared text between chunks to preserve context at boundaries.

44. What is reranking?
A second ranking step to reorder retrieved chunks by final relevance.

45. How handle poor retrieval?
Ask clarification, lower confidence, avoid pretending context exists, and cite only real sources.

46. What does a report include?
Risk summary, precautions, diet, wellness plan, doctor questions, reminders, and sources.

47. Why mandatory fields?
Reports require enough structured data to avoid unsafe generic output.

48. Why consent validation?
Health data processing must be explicit and auditable.

49. What is triage?
Rule-based urgent symptom and risk detection before LLM/report generation.

50. What is risk scoring?
Deterministic scoring across symptoms, lifestyle, conditions, sleep, diet, and red flags.

51. How do you handle PDF generation?
The report/PDF layer formats structured report data into a doctor-ready document with sections, sources, review metadata, and page numbers.

52. How do you handle doctor review?
Doctors review submitted reports, add status, notes, priority, signature, and follow-up guidance without bypassing patient ownership controls.

53. How do you handle admin knowledge?
Admins upload trusted knowledge with category and citation metadata so RAG can retrieve approved context.

54. How do you handle compliance role?
Compliance users inspect audit logs, privacy controls, security status, and operational evidence.

55. How do you handle email verification?
Registration creates an OTP; the account stays restricted until the email is verified.

56. How do you handle Google SSO?
The app has provider start/callback scaffolding so production identity can integrate without changing business flows.

57. How do you handle backend route protection?
Every protected API validates the token, role, and ownership. Frontend hiding is never trusted.

58. How do you handle sessions?
Sessions or tokens identify verified users and are checked on each protected request.

59. How do you handle CSRF?
Cookie-authenticated writes should require a CSRF token so other sites cannot submit actions as the user.

60. How do you handle secret management?
Local .env is for demo only. Production should use a cloud secret manager or vault.

61. How do you handle SPF?
SPF authorizes which mail servers can send email for the domain.

62. How do you handle DKIM?
DKIM signs email so receivers can verify it came from an authorized domain sender.

63. How do you handle DMARC?
DMARC tells receivers what to do when SPF or DKIM fails and improves email trust.

64. How do you handle no personal Gmail?
Production should use a domain no-reply sender with SPF, DKIM, DMARC, auditability, and deliverability.

65. How do you handle PHI masking?
Logs should avoid raw health text and store IDs, hashes, event types, and limited metadata.

66. How do you handle audit logging?
Important actions such as login, upload, report generation, review, and deletion are recorded.

67. How do you handle privacy export?
Users can export their stored reports, chats, consents, and related health data.

68. How do you handle delete health data?
A confirmation flow deletes user-owned reports, chats, documents, and profiles.

69. How do you handle voice silence detection?
A timer resets when speech arrives and auto-stops after five seconds of silence.

70. How do you handle interim transcripts?
Interim results make speech capture feel live and prevent losing partial phrases.

71. How do you handle manual stop?
Manual Stop lets the user finish immediately without waiting for silence detection.

72. How do you handle voice assessment separation?
Voice questions go to lightweight chatbot mode unless the user explicitly starts assessment/report generation.

73. How do you handle voice follow-ups?
The bot asks one short question at a time and limits follow-ups to a few useful details.

74. How do you handle voice summary?
After enough details, the bot summarizes symptoms and gives precautions, diet, hydration, rest, and doctor-prep notes.

75. How do you handle safe wording?
The assistant uses calm wording such as consult a clinician or seek urgent care for specific red flags.

76. How do you handle allergy extraction?
The NLP layer scans allergy/allergic phrases and stores mentioned allergy context.

77. How do you handle age extraction?
The NLP layer detects phrases such as age is, I am, or I'm followed by a number.

78. How do you handle severity extraction?
Severity can be mild/moderate/severe, a 1-10 score, or a high temperature value.

79. How do you handle duration extraction?
Duration is extracted from phrases like for two days, since yesterday, today, or for 3 hours.

80. How do you handle medicine extraction?
Medicine intent is detected from medicine, medication, tablet, antibiotic, can I take, and similar wording.

81. How do you handle condition extraction?
Conditions such as diabetes, BP, asthma, thyroid, cholesterol, kidney, and heart disease are recognized.

82. How do you handle follow-up selection?
Missing required fields for the detected intent decide the next single follow-up question.

83. How do you handle default answer style?
The chatbot defaults to 1-3 short lines and avoids long paragraphs unless requested.

84. How do you handle streaming responses?
Streaming sends token-like chunks so users see progress before the final answer is complete.

85. How do you handle rate limiting?
Rate limits protect APIs from abuse and accidental repeated requests.

86. How do you handle upload scanning?
Uploads are checked for size, type, suspicious content, and extraction viability before indexing.

87. How do you handle unsupported browser voice?
The UI shows a clear message and allows manual transcript typing.

88. How do you handle ASR failure?
The backend returns a safe generic transcription error without leaking provider internals.

89. How do you handle local vector store?
The demo can store embeddings as JSON locally for simple semantic retrieval.

90. How do you handle pgvector?
pgvector is useful when PostgreSQL is already the primary database and vector search should stay nearby.

91. How do you handle Qdrant?
Qdrant is suitable for a dedicated scalable vector service with payload filtering and fast retrieval.

92. How do you handle prompt output tests?
Prompt regression tests assert style, refusal language, shortness, stages, and no unsafe content.

93. How do you handle RAG evaluation?
RAG evals check that known queries retrieve expected chunks and cite correct sources.

94. How do you handle backups?
Production needs database, object storage, vector index, and audit log backups with restore drills.

95. How do you handle rollback?
Rollback returns the system to a known-good deployment if smoke tests or metrics fail.

96. How do you handle model upgrades?
Model upgrades require prompt regression, safety, RAG, and cost/latency tests before release.

97. How do you handle token optimization?
Keep prompts short, retrieve only relevant chunks, limit memory, and choose compact output.

98. How do you handle small classifier model?
A smaller model or rules can classify intent cheaply before calling a larger model.

99. How do you handle large final model?
A larger model is reserved for final synthesis when context and safety checks justify it.

100. How do you handle hallucination rate?
Track unsupported claims and improve prompts/retrieval when the rate rises.

32. Final Summary

HealthGuard AI is strong because it combines healthcare safety, RAG, voice, NLP intent understanding, memory isolation, role-based access, document processing, report generation, doctor review, audit logging, and regression testing. It is demo-ready and has a clear roadmap for PostgreSQL, pgvector/Qdrant, Redis/Celery, object storage, HTTPS, SIEM, secret managers, and formal compliance controls.

- Explain it confidently from user workflow to backend architecture.
- Emphasize that LLM is not the safety engine.
- Tie every design decision to safety, scalability, maintainability, or user trust.